

Automação inteligente e dados híbridos

Relatório técnico do desafio Ir Além 2

Felipe Livino dos Santos | RM 563187; Daniel Veiga Rodrigues de Faria | RM 561410; Tomas Haru Sakugawa Becker | RM 564147; Daniel Tavares de Lima Freitas | RM 562625; Gabriel Konno Carrozza | RM 564468.

1 Objetivo e recorte da automação

O segundo desafio adicional do Cardiola demonstra um fluxo periódico que lê medições fictícias, aplica uma técnica de detecção de anomalias e registra o resultado de forma rastreável. Também incorpora mensagens textuais em um banco documental e reutiliza o serviço de extração generativa para interpretá-las. A automação reúne, assim, processamento numérico, interpretação de texto e persistência em dois tipos de banco.

A proposta foi implementada em Python, com execução agendada pelo próprio processo do robô. Não se trata de um robô que imita cliques em uma interface. O processo automatiza etapas de leitura, análise, gravação e sincronização, conforme o recorte de RPA descrito no enunciado. Os dados são simulados, e os alertas indicam desvios estatísticos em relação a uma base artificial, sem classificar risco clínico ou recomendar conduta.

As escolhas principais foram SQLite para registros estruturados, MongoDB para documentos e eventos e Isolation Forest para identificar observações que diferem do conjunto de referência. Essas tecnologias estão presentes no capítulo 2 de AIRPA. O processamento textual aproveita a integração generativa desenvolvida no primeiro desafio, evitando manter duas implementações de extração com regras diferentes [1].

2 Organização dos componentes

O Docker Compose define a aplicação web, o MongoDB e o robô. Os dois últimos são ativados pelo perfil automacao. A aplicação mostra as execuções no painel Monitoramento; o robô realiza os ciclos; o MongoDB armazena mensagens e a cópia dos eventos. Como o SQLite é embutido no Python, não existe um container dedicado a esse banco. Seus arquivos ficam no volume persistente compartilhado entre os processos que precisam consultá-los.

Componente	Responsabilidade
Robô Python	Executar o ciclo, validar medições, aplicar o modelo, gravar resultados e tratar pendências.
SQLite	Manter medições, avaliações, alertas, execuções e a fila local de sincronização.
MongoDB	Armazenar mensagens textuais, eventos e documentos das execuções.
Serviço Gemini	Extrair fatos de mensagens pendentes com evidência literal e controle de chamadas.

O MongoDB fica em uma rede interna do Compose, sem porta publicada no computador. A inicialização do robô depende da verificação de saúde do banco. Essa dependência organiza a partida dos serviços, mas não elimina falhas durante o funcionamento. Por esse motivo, o ciclo mantém uma fila persistente que permite reconciliar os eventos quando a conexão volta.

3 Preparação dos dados e escolha dos atributos

A carga inicial usa uma semente fixa e o identificador fictício SIM-001. São 240 medições destinadas ao treinamento e 63 destinadas ao monitoramento. O campo dataset separa os grupos, de modo que o modelo não seja treinado com os mesmos registros usados para demonstrar a análise posterior. Essa separação melhora a clareza do experimento, embora os dois grupos continuem pertencendo a um cenário artificial.

Cada medição contém pressão sistólica, pressão diastólica, frequência cardíaca e adesão. A adesão é representada por um número entre zero e um, conforme a codificação simulada do conjunto. Não há cálculo de dose nem avaliação do tratamento. Os dados foram escolhidos para demonstrar o processamento e não devem ser interpretados como faixas de normalidade ou parâmetros clínicos.

Três registros funcionam como casos controlados: o 301 contém um desvio inserido intencionalmente; o 302 representa uma observação de referência; o 303 contém um dado ausente. Eles permitem verificar caminhos diferentes sem depender apenas da contagem final de alertas. A carga é idempotente, isto é, pode ser preparada novamente sem multiplicar os mesmos registros.

4 Validação e detecção com Isolation Forest

Antes da análise, o código confere se os quatro atributos são numéricos e finitos. Valores booleanos não são aceitos como números. As três medidas fisiológicas precisam ser positivas, e a adesão deve estar no intervalo definido. Um registro inválido recebe o rótulo dado_invalido e não é enviado ao modelo. Isso distingue um problema de qualidade de dados de uma anomalia calculada a partir de valores válidos.

O treinamento exige pelo menos 30 amostras válidas e usa Isolation Forest com 100 árvores, contamination igual a 0,08 e random_state igual a 42. A técnica isola observações por partições sucessivas; neste projeto, a predição -1 é tratada como desvio em relação ao conjunto sintético. O parâmetro de contaminação define o corte utilizado no treino. Ele não representa prevalência de doença nem garante que exatamente 8% das medições de monitoramento sejam marcadas.

Para cada medição válida, a aplicação conserva a classificação e o escore de decisão. O código também calcula uma versão do modelo a partir dos parâmetros, dos atributos, dos valores de treino e da versão da implementação. Esse identificador ajuda a saber sob qual configuração uma avaliação foi produzida. Uma nova versão pode reavaliar a mesma medição sem apagar o resultado anterior.

Não foram calculadas sensibilidade, especificidade ou outras medidas de desempenho clínico. O experimento verifica a coerência do processamento dentro da base simulada. Essa distinção é necessária porque uma técnica de detecção de anomalias pode funcionar corretamente no código e, ainda assim, não ter utilidade demonstrada em dados reais.

5 Etapas de um ciclo de execução

Cada ciclo começa com a criação de um identificador único e com o registro do horário e da situação `em_execucao`. Em seguida, o robô prepara a carga inicial, treina o modelo com o grupo de treino e consulta as medições de monitoramento que ainda não possuem avaliação para aquela versão. Essa consulta evita processar novamente o mesmo par de medição e modelo.

O processamento percorre as medições, valida os atributos e calcula a classificação quando isso é possível. Em uma transação local por medição, grava a avaliação e cria o evento correspondente. Quando o resultado é `anomalia_estatistica`, também grava um alerta. A avaliação de um dado inválido gera um evento próprio; ela não é contada como alerta estatístico.

Depois da etapa numérica, o robô tenta sincronizar a fila local com o MongoDB. Também prepara as mensagens de demonstração sem duplicá-las e procura textos ainda pendentes. Em cada ciclo, a consulta limita esse processamento a até cinco mensagens. A interpretação usa o mesmo serviço Gemini da interface, com o mesmo esquema, cache e controle de tentativas. Quando uma mensagem é interpretada, o documento conserva os fatos, o texto e o vínculo com a execução.

Uma falta de chave ou de disponibilidade para extração não deve apagar o trabalho numérico realizado. A execução registra sua situação e mantém as mensagens pendentes. Ao final, o robô grava o horário de término, o número de medições processadas e eventual código de erro, e tenta novamente escoar a fila. O intervalo padrão entre ciclos é de 60 segundos, configurável por variável de ambiente.

Situação	Leitura operacional
concluida	O ciclo terminou sem a condição de pendência ou falha registrada nos demais estados.
aguardando_gemini ou parcial	A etapa textual depende de configuração ou encontrou uma falha; mensagens podem continuar pendentes.
falha	Uma exceção interrompeu parte do ciclo e foi registrada de forma controlada.
sincronizacao_pendente	A tentativa final de replicar eventos não foi concluída; a fila local permite tentar novamente.

Um bloqueio de arquivo no volume impede a execução simultânea de dois robôs sobre a mesma base. Para rodar um ciclo isolado com a opção `--once`, o procedimento documentado pede parar primeiro o processo periódico. O encerramento normal preserva os volumes e, portanto, o histórico necessário para retomar o processamento.

6 Modelo de dados e rastreabilidade

O SQLite foi usado para os dados que possuem relações e regras estáveis. A tabela `measurements` associa cada medição ao paciente fictício e ao conjunto de treino ou monitoramento. `evaluations` usa a combinação de medição e versão do modelo como chave, enquanto `alerts` mantém um identificador estável por resultado. A tabela `runs` permite recuperar a situação e a quantidade processada em cada ciclo.

Estrutura	Informação preservada
patients e measurements	Identificação fictícia, horário, atributos numéricos e separação dos conjuntos.
evaluations e alerts	Classificação, escore, versão do modelo, execução de origem e motivo do alerta.
runs	Início, término, situação, quantidade processada e código de erro.
outbox	Identificador do evento, coleção de destino, conteúdo JSON e indicador de entrega.
MongoDB messages, events e runs	Textos, resultados de extração, eventos de análise e documentos das execuções.

O banco documental permite manter o texto original junto com os fatos extraídos e metadados que variam conforme o evento. Ele não substitui a origem relacional das medições. A associação entre `measurement_id` ou `message_id`, `run_id`, modelo e horário torna possível percorrer o caminho de um evento até o registro que o produziu. Rastreabilidade, neste caso, significa conseguir explicar o que foi processado e em qual execução.

7 Sincronização e recuperação de falhas

A fila `outbox` resolve uma situação importante: a análise pode ser gravada com sucesso no SQLite enquanto o MongoDB está indisponível. Em vez de perder o evento, o robô o mantém localmente com o indicador de entrega pendente. A rotina de sincronização envia cada item por `upsert`, utilizando o mesmo identificador, e só marca a entrega depois da resposta bem-sucedida do banco documental.

Se o processo falhar depois de enviar o documento, mas antes de marcar a entrega local, o item será reenviado. Como o identificador é estável, a operação substitui o documento correspondente em vez de inserir uma cópia. Isso permite repetir a sincronização sem duplicar eventos. A mesma ideia aparece nas avaliações e nos alertas, que possuem chaves estáveis no banco relacional.

A solução não implementa uma transação distribuída entre SQLite e MongoDB. Durante uma falha, os dois bancos podem mostrar estados diferentes até a reconciliação. O contador de pendências torna essa condição observável. O estado de uma execução que falhou permanece no histórico, mesmo depois que a fila foi esvaziada por um ciclo posterior; assim, a recuperação não apaga a evidência do problema.

8 Resultados e verificação do fluxo

A execução documentada em 4 de setembro de 2026 avaliou as 63 medições de monitoramento, gerou 11 alertas estatísticos e terminou com zero eventos pendentes. Os 240 registros de treino permaneceram separados. As três mensagens de demonstração no MongoDB foram interpretadas pelo Gemini e vinculadas às execuções. Esses números descrevem a base artificial utilizada; não expressam taxa de doença, precisão clínica ou desempenho em uma população [2].

Cinco testes automatizados do robô verificaram os casos controlados, a repetição de processamento sem duplicação e a recuperação da fila após falha simulada. Além disso, a documentação registra uma verificação com o MongoDB desligado: o robô marcou `sincronizacao_pendente` e conservou os eventos. Depois do retorno do banco, uma nova execução reconciliou a fila sem duplicar os 11 alertas. A recriação dos containers também preservou os dados armazenados nos volumes.

É importante separar esse experimento histórico de uma leitura atual do painel. O conjunto inicial é fixo; o robô não inventa novas medições a cada minuto. Por isso, ciclos posteriores podem apresentar zero registros processados e ainda estar funcionando corretamente. Eles continuam verificando textos e eventos pendentes. Para demonstrar uma nova análise numérica, é necessário inserir uma medição adicional pelo módulo previsto no projeto.

9 Operação e decisões de projeto

A execução completa usa “`docker compose --profile automacao up --build -d`”. A aba Monitoramento apresenta as quantidades de medições, alertas e eventos pendentes, além das execuções recentes. A interface mostra datas no horário local do navegador e nomes de situações mais legíveis. Para uma análise técnica, os logs do serviço robot e o módulo `automation.inspect_data` permitem consultar o processamento sem alterar a base.

Optamos por manter as falhas visíveis e recuperar o fluxo por repetição segura, em vez de depender de uma única gravação bem-sucedida em todos os componentes. Essa decisão torna o exemplo um pouco mais extenso, mas permite demonstrar por que uma automação precisa de estado persistente. A combinação de transações locais, identificadores estáveis e fila de eventos é o que permite retomar o trabalho sem perder sua origem.

10 Limitações e conclusão

O robô atende ao recorte acadêmico de leitura periódica, uso de IA, dados híbridos e registro rastreável. Seus limites incluem a base pequena e sintética, o processamento por um único robô e a dependência de configuração e cota para interpretar textos. Não foram demonstradas operação em grande escala, validação clínica ou implantação com dados reais. Uma evolução deve ampliar os testes de falha, avaliar outras distribuições de dados e estabelecer critérios de qualidade antes de expandir o uso.

[1] FIAP. Do Banco de Dados à Automação Inteligente. AIRPA, Fase 5, capítulo 2, p. 9 a 11, 23 a 30 e 46 a 48. Material didático da atividade.

[2] Projeto CardiolA. docs/evidence/automation-run.json e docs/VALIDACAO.md. Verificação registrada em 4 set. 2026. Código e estruturas: automation/robot.py, automation/analysis.py, database/relational/schema.sql e database/nonrelational/README.md.